Module 02 — Modules and Packages

Phase: Advanced Python
Previous: Functions, Decorators, and Generators
Next: OOP

Module Outcome

Split reusable functions into importable files, isolate libraries with a virtual environment, structure a small package, and use a few standard-library modules.

By the end of this module you will be able to:

- Create and activate a `venv`, then freeze installs to `requirements.txt`
- Import a module (`import math`, `from math import sqrt`, aliases)
- Build a folder package with `__init__.py` and import helpers from `main.py`
- Guard demos with `if __name__ == "__main__"`
- Use `datetime`, `json` (strings), `os.path.join`, and `random` for small jobs

---

1. Why This Concept Matters

You can now write functions. Leave them all in one file and the script grows until nobody can find `is_positive`. A **module** is one `.py` file of related names. A **package** is a **folder** of modules (usually with `__init__.py`).

Unit	Meaning
Function	A named group of statements
Module	Related functions in one `.py` file
Package	A folder of modules you import as a group

`import` reuses that work without copy-paste. Two projects on one lab PC also need their **own** library versions — that sandbox is a **venv**.

Tonight's mini-project kit (venv package bootstrap) is this layout: a small `campus_utils` package plus an isolated environment.

---

2. Virtual environment (`venv`)

Understand

A **venv** is a project-private copy of Python plus whatever you `pip install`. Project A and Project B can then use different library versions on the same machine.

pip installs packages from PyPI into the **active** environment. Teammates recreate that set from `requirements.txt`.

How It Works

```
python -m venv venv

# Activate (pick your OS):
source venv/bin/activate          # macOS / Linux
# venv\Scripts\activate           # Windows Command Prompt
# venv\Scripts\Activate.ps1       # Windows PowerShell

pip install requests
pip freeze > requirements.txt
pip install -r requirements.txt   # teammate / new machine
deactivate
```

After activate, `pip` and `python` point at `venv/`, not the system Python. You do not need a live HTTP call to prove the venv works — `pip freeze` listing what you installed is enough.

⚠️ WATCH OUT

```
# Wrong — commit the venv folder (huge, OS-specific paths)
git add venv/

# Prefer
# add venv/ to .gitignore
# commit requirements.txt only

# Wrong — pip install with no venv active (pollutes the machine)
# Prefer — activate first, then pip install
```

Try It

In an empty practice folder: create a venv, activate it, run `pip freeze`, then `deactivate`. Confirm the prompt changes when the venv is on.

---

3. Import

Understand

`import` loads another module's names into the current file.

Form	Call style
`import math`	`math.sqrt(16)` — prefix shows where it came from
`from math import sqrt, pi`	`sqrt(16)` — shorter
`import datetime as dt`	alias for a long module name
`from math import *`	**avoid** — hides where names came from

Python searches, in order: the current directory, `PYTHONPATH`, the standard library, then `site-packages` (pip installs). Wrong folder or a typo → `ModuleNotFoundError`. Run from the project root; match the filename exactly (Linux is case-sensitive).

`import sys` then `sys.path` is the debug list if an import fails — you do not need it for everyday work.

How It Works

```
import math
print(math.ceil(4.2))
print(math.sqrt(81))

from math import ceil, sqrt
print(ceil(4.2))
print(sqrt(81))
```

Expected Output:

```
5
9.0
5
9.0
```

```
import datetime as dt

stamp = dt.datetime(2026, 9, 14, 18, 30)
print(stamp.strftime("%d-%m-%Y %H:%M"))
```

Expected Output:

```
14-09-2026 18:30
```

Your own `greetings.py` in the same folder imports the same way (`from greetings import namaste`). If the file is missing or you run from the wrong folder: `ModuleNotFoundError`.

⚠️ WATCH OUT

```
# Wrong
from math import *

# Prefer
import math
from math import sqrt
```

Try It

Print `math.floor(4.9)` with a module prefix, then import `floor` by name and print `floor(4.9)` with no prefix.

---

4. Packages

Understand

A **package** is a folder you import. `__init__.py` (even empty) marks it as a package and can re-export names so `main.py` stays short.

```
my_project/
├── venv/                # not committed
```

```
├── requirements.txt
├── main.py
└── campus_utils/
    ├── __init__.py
    └── formatters.py
```

- **Absolute** import — full path from the project: `from campus_utils.formatters import format_inr`
- **Relative** import — from **inside** the package: `from .formatters import format_inr` (`.` = this package)

Use absolute imports from `main.py`. Use relative imports in `__init__.py` and sibling package files. Relative imports fail if you run the file as a loose script (`python formatters.py`).

How It Works

```
# campus_utils/formatters.py
def format_inr(n):
    return f"₹{n:,.2f}"
```

```
# campus_utils/__init__.py
from .formatters import format_inr

__all__ = ["format_inr"]
```

```
# main.py  (project root — run: python main.py)
from campus_utils import format_inr

print(format_inr(1250))
```

Expected Output:

```
₹1,250.00
```

Without the re-export you would write `from campus_utils.formatters import format_inr` — also correct, just longer.

⚠️ WATCH OUT

```
# Wrong — relative import from a loose script
python formatters.py          # from .x import y  fails

# Prefer
python main.py                # main uses absolute imports

# Missing folder marker (older tools / some layouts)
# add campus_utils/__init__.py
```

Circular imports (`a` imports `b` imports `a`) fail or partially initialize. Move shared helpers to a third module.

Try It

Create `campus_utils/formatters.py` with `format_inr`. Re-export it from `__init__.py` with a relative import. From `main.py` print `format_inr(99)`.

---

5. `if __name__ == "__main__"`

Understand

Every module has a `__name__`.

- You **run** the file (`python greetings.py`) → `__name__` is `"__main__"`
- Another file **imports** it → `__name__` is `"greetings"`

Put demos and CLI entry points under `if __name__ == "__main__":` so `import greetings` does not print or wait for `input`.

How It Works

```
# greetings.py
def namaste(name):
    return f"Namaste, {name}!"

if __name__ == "__main__":
    print("Running as script:", namaste("Ravi"))
```

```
# main.py
import greetings

print(greetings.namaste("Priya"))
```

Run `python main.py`:

```
Namaste, Priya!
```

No `"Running as script"` line — the guard skipped the demo.

Run `python greetings.py`:

```
Running as script: Namaste, Ravi!
```

⚠ WATCH OUT

```
# Wrong — top-level print / input runs on every import
print(namaste("Priya"))

# Prefer
if __name__ == "__main__":
    print(namaste("Priya"))
```

Try It

Move a demo `print` under the guard in `greetings.py`. Import `namaste` from `main.py` and confirm the demo line does **not** appear.

---

6. Standard library highlights

Understand

The **standard library** ships with Python — no `pip` needed. Prefer it before adding a third-party package for a simple job.

Module	Job	Typical calls
`math`	Numbers	`sqrt`, `ceil`, `floor` (already used above)
`datetime`	Dates / times	`datetime(...)`, `strftime`, `timedelta`
`random`	Demo picks / OTPs	`randint`, `choice`, `sample` — not for real secrets
`json`	Dict ↔ JSON **string**	`dumps`, `loads`
`os`	Folders / env	`getcwd`, `path.join`, `environ.get`

JSON **files** (`dump` / `load` with `open`) wait for Files week. Here you only round-trip a **string**.

How It Works

```
from datetime import datetime, timedelta

start = datetime(2026, 9, 14, 18, 30)
expiry = start + timedelta(days=30)
print("Expires on:", expiry.date())
```

Expected Output:

```
Expires on: 2026-10-14
```

```
import json

batch = {"city": "Pune", "batch": "PFS-01"}
text = json.dumps(batch)
print("JSON string:", text)
data = json.loads(text)
print("City:", data["city"])
```

Expected Output:

```
JSON string: {"city": "Pune", "batch": "PFS-01"}
City: Pune
```

`json.dumps` / `loads` are the **string** pair. `dump` / `load` need a file object — later.

```
import os

print(os.environ.get("DB_HOST", "localhost"))
print(os.path.join("data", "reports"))  # `\` on Windows, `/` on macOS/Linux
```

Expected Output (first line; the path shape follows your OS):

```
localhost
```

```
import random

print(random.randint(100000, 999999))
print(random.choice(["Canteen", "Mess", "Home"]))
```

The numbers change each run. That is fine for a class demo. Real login OTPs belong in `secrets` later — not this module.

⚠️ WATCH OUT

```
# Wrong — glue paths with +
"data/" + "reports"           # breaks on Windows vs macOS

# Prefer
os.path.join("data", "reports")

# Wrong — treat classroom random OTP as security
# Prefer secrets for anything real (later)
```

Try It

Dump `{"city": "Hyderabad", "batch": "PFS-01"}` with `json.dumps`. Load it back and print the city.

Step-by-Step: Build It

One assemble: a quiet helper module, a `__main__` demo, and a `main.py` that imports it. Create both files in the **same folder**, then run from that folder.

File `greetings.py`:

```
APP_VERSION = "1.0.0"

def namaste(name):
    return f"Namaste, {name}!"

if __name__ == "__main__":
    print("Running as script:", namaste("Ravi"))
```

File `main.py`:

```
import greetings
```

```
print(greetings.namaste("Priya"))
print("Version:", greetings.APP_VERSION)
```

Expected Output (`python main.py`):

```
Namaste, Priya!
Version: 1.0.0
```

Expected Output (`python greetings.py`):

```
Running as script: Namaste, Ravi!
```

Next step in the mini-project: the same idea as a **package** (`campus_utils/`) inside a **venv**.

Practice Exercises

Attempt each problem before reading the model answer.

1. Import `math`. Print `math.pow(2, 3)` and `math.pi` rounded to 2 decimals with `round(math.pi, 2)`.

Model answer:

```
import math

print(math.pow(2, 3))
print(round(math.pi, 2))
```

Expected Output:

```
8.0
3.14
```

2. Round-trip a dict with `json.dumps` / `json.loads` (no file). Print the JSON string and then `data["batch"]`.

```
record = {"city": "Hyderabad", "batch": "PFS-01"}
```

Hint: `loads` parses a **string**, not a file.

Model answer:

```
import json

record = {"city": "Hyderabad", "batch": "PFS-01"}
text = json.dumps(record)
print(text)
data = json.loads(text)
print(data["batch"])
```

Expected Output:

```
{"city": "Hyderabad", "batch": "PFS-01"}
PFS-01
```

3. Print 6 unique lottery numbers from `1` through `49` with `random.sample`, then sort them. (Values change each run.)

Hint: `random.sample(range(1, 50), 6)`.

Model answer:

```
import random

print(sorted(random.sample(range(1, 50), 6)))
```

4. `date_utils.py` defines `format_iso(year, month, day)` and prints a demo under `if __name__ == "__main__":`. Another file does `from date_utils import format_iso` and prints `format_iso(2026, 9, 14)`. What extra line appears, if any, when you run the **importing** file?

Hint: Same idea as the `greetings` guard.

Model answer:

No demo line. `__name__` inside `date_utils` is `"date_utils"`, not `"__main__"`. The print is `2026-09-14` only.

5. Sketch the folders for a package `inventory/` with `products.py` (`make_product(sku, name, price)` returning a dict) and `__init__.py` that relative-imports `make_product`. Write the one import line `main.py` should use.

Hint: No classes — a product is a dict, as in Core Python.

Model answer:

```
inventory/
  __init__.py      # from .products import make_product
  products.py
main.py            # from inventory import make_product
```

```
def make_product(sku, name, price):
    return {"sku": sku, "name": name, "price": price}
```

6. A teammate cloned your repo and has no `venv`. List the commands (macOS/Linux **or** Windows) to create a venv, activate it, and install from `requirements.txt`. Why is `venv/` not in Git?

Model answer:

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

`venv/` is large and OS-specific. `requirements.txt` is the recipe; each machine builds its own sandbox.

Lab

Combine a local module, an import, and the `__main__` guard. No venv required in class. No `open()`, no CSV.

Problem

`namaste` is stuck in a script that prints as soon as anyone imports it.

What to Build

Two files in one folder: a quiet helper module and a small driver.

Requirements

- `greetings.py` — `namaste(name)` returns a string; demo print only under `if __name__ == "__main__":`
- `main.py` — import and print `namaste("Priya")` (or another name)
- Running `main.py` must **not** print the demo line from `greetings.py`
- Running `greetings.py` **does** print the demo

Expected Behavior

Import stays quiet. Direct run shows the demo.

Sample Run

```
$ python main.py
Namaste, Priya!

$ python greetings.py
Running as script: Namaste, Ravi!
```

Constraints

No packages required. No `try` / `except`. No files other than the two `.py` modules.

Bonus

Add `APP_VERSION` in `greetings.py` and print it from `main.py`.

Mini-Project (Assignment): Project Bootstrap Template

Why this matters: Later CLI and file apps start as a folder: venv, a small package, `main.py`, and a README a teammate can follow.

Lab kit: 15_Lab_Assets/16_venv_package_bootstrap/

File	What you do
`starter/campus_utils/formatters.py`	Finish `format_inr`
`starter/campus_utils/__init__.py`	Re-export so `from campus_utils import format_inr` works
`starter/main.py`	Already imports — run it inside an activated venv
`broken/`	Fix the import / formatter bug

File	What you do
`solution/`	Do **not** copy this

Optional polish with the toolkit from last module: 30_ksr_utils_toolkit.

This is **not** the two-file lab. Same import ideas, plus isolation and a package layout.

Objective

Ship a `py_starter`-style skeleton: venv + `campus_utils` + `main.py` + README.

Story / Scenario

A teammate clones your repo on a lab PC. They should recreate the environment from `requirements.txt` and run `main.py` without `sys.path` hacks.

Requirements

Piece	Detail
`venv/` + `requirements.txt`	Isolated env; freeze after any installs
`campus_utils/`	`__init__.py` + at least `formatters.py`
`main.py`	Imports package helpers; prints a short demo
README	Activate, install, run — Windows and macOS/Linux

A small JSON defaults file is **optional stretch** (Files week will go deeper). Do not spend the core ticket on `open()`.

Expected Behavior

- `format_inr` prints a ₹ amount from `main.py`
- Package import works from the project root

Sample Run

```
$ source venv/bin/activate
$ python main.py
₹1,234.50
```

Bonus Challenges

- Second helper module + relative re-export in `__init__.py`
- Pin versions in `requirements.txt` (`package==1.2.3`)
- Optional: load a tiny JSON defaults **string** or, after Files week, `data/sample.json`

Done when

- [ ] `starter/main.py` runs inside an activated venv
- [ ] Package import works without `sys.path` hacks
- [ ] README shows recreate-from-`requirements.txt`
- [ ] No `venv/` committed when you push

Interview Q&A

Q1. Module vs package?
A module is one `.py` file. A package is a folder of modules (typically with `__init__.py`) imported as a hierarchy, e.g. `campus_utils.formatters`.

Q2. Why `venv`?
It isolates pip installs per project so two apps on one machine do not fight over library versions. Commit `requirements.txt`, not `venv/`.

Q3. What is `if __name__ == "__main__":` for?
That block runs only when the file is executed as a script, not when imported. Keeps modules reusable and quiet.

Q4. `json.dumps` vs `json.dump`?
`dumps` / `loads` work on **strings**. `dump` / `load` work on **file objects** (Files week). Prefer the string pair until you have `open`.

Q5. Absolute vs relative import?
Absolute uses the full package path (`from campus_utils.formatters import format_inr`) — use it from `main.py`. Relative uses `.` / `..` from the current package file — common in `__init__.py`. Relative imports fail on a loose `python some_module.py`.

Q6. Why `os.path.join` instead of `"folder/" + "file"`?
Separators differ on Windows vs macOS/Linux. `join` builds a path for the current OS.

Q7. How do you share dependencies?
`pip freeze > requirements.txt`. Teammates activate their venv and run `pip install -r requirements.txt`.

Q8. Why prefer `import json` over writing your own parser?
JSON is a standard format. The stdlib module is tested, handles types like `true` / `null`, and is what APIs expect.

Cheat Sheet

```
python -m venv venv     → create sandbox
source venv/bin/activate → macOS/Linux on
venv\Scripts\activate   → Windows on
pip freeze > requirements.txt
pip install -r requirements.txt
deactivate              → leave sandbox

import math             → math.sqrt
from math import sqrt   → sqrt (named)
import datetime as dt   → alias
from math import *      → avoid

from pkg import name    → package re-export
from .mod import name   → relative (inside package only)

if __name__ == "__main__":
                        → demo / CLI; skipped on import

datetime.strftime       → date text
json.dumps / loads      → dict ↔ JSON string
os.path.join(...)       → portable path
random.choice / sample  → demo picks (seed for class)
```

Completion Checklist

```
[ ] I can create, activate, and freeze a venv
[ ] I can import a stdlib module and my own .py file
[ ] I can explain ModuleNotFoundError (folder / name)
[ ] I can layout a package with __init__.py
[ ] I can use a relative import inside the package
[ ] I can guard demos with if __name__ == "__main__"
[ ] I can round-trip a dict with json.dumps / loads
[ ] I completed the two-file lab
[ ] I attempted the venv + campus_utils mini-project
[ ] I can explain module vs package in an interview
```

What's Next?

You can now split functions across files and keep each project's libraries in a sandbox.

Next, OOP groups **data and behaviour** into classes — an account is no longer a loose `balance` plus functions; it becomes an object with methods.